Speech Understanding PA-2: Code-Switched Lecture Transcription, Translation to Maithili, and Voice Cloning with Adversarial Robustness Analysis

Rahul Agarwal (Roll No. M25DE1040)

Speech Understanding | PA-2 | Instructor: Prof. Richa Singh | Code: github.com/rahulagarwal2011/Speech_Assignment_2

Abstract—We present an end-to-end pipeline for processing a 10-minute code-switched (Hindi/English) classroom lecture and re-synthesising it in Maithili. The system has four parts: (I) speech-to-text with a frame-level CNN+BiGRU language identifier, a script-based dual-pass code-switch detector, and Whisper large-v3 under *n*-gram-biased constrained decoding; (II) IPA conversion via epitran and neural translation by IndicTrans2 backed by a 511-term technical dictionary spread across 8 domains; (III) zero-shot voice cloning with a dual-model TTS strategy (Coqui XTTS v2 and Meta MMS-TTS) followed by Sakoe–Chiba-band DTW prosody warping using the WORLD vocoder; and (IV) LFCC+LCNN anti-spoofing detection plus FGSM adversarial robustness analysis with binary search for the smallest imperceptible perturbation. Reported numbers come from a single full pipeline run on an Apple M4 Pro: 88 transcribed segments (86 Hindi, 2 English), WER = 21.7% (English) and 86.1% (Hindi, inflated by ground-truth misalignment), MCD 10.19 dB (MMS) vs. 14.70 dB (XTTS), anti-spoofing EER = 0.0%, FGSM flip at $\varepsilon = 1.0\times10^{-4}$ with SNR = 61.5 dB, and frame-level LID F1 = 1.0 on a held-out En/Hi validation split. Three contributions are formalised: a closed-form *n*-gram logit biasing rule, an honest ablation of DTW prosody warping versus flat synthesis (warping slightly increases evaluation MCD by 0.21 dB), and a frame-level LID confusion matrix together with a code-switch boundary error decomposition (4 boundaries detected of 16 ground-truth).

Index Terms—code-switching, ASR, constrained decoding, IndicTrans2, voice cloning, DTW, LCNN, FGSM.

I. INTRODUCTION

Real-world classroom speech in India is heavily code-switched: a lecturer freely mixes English technical terms into Hindi explanations and occasionally drops into a regional language for clarification. Off-the-shelf monolingual ASR and TTS stacks fragment on these transitions, and Whisper's own language detector tends to label Hindi-dominant Hinglish as English (and, when forced into Hindi mode, translates rather than transcribes the Devanagari content). The objective of this work is to build a fully-local pipeline that (a) transcribes such a lecture without any cloud APIs, (b) translates it into Maithili while preserving domain terminology, and (c) re-synthesises the audio in the student's own voice with the professor's prosody transferred on top. We operate on a single 10-minute recording covering 88 segments (86 Hindi, 2 English) and we treat every design choice as a research question whose answer is logged.

Three components are formalised here. Section III derives the constrained-decoding rule used on top of Whisper large-v3, which combines an external trigram language model with a hard technical-vocabulary boost. Section IV reports the prosody ablation in which the same Maithili utterances are resynthesised with and without DTW warping, and includes the dual-model TTS comparison between XTTS v2 and MMS-TTS. Section V introduces a frame-level confusion matrix for the CNN+BiGRU language classifier together with the code-switch boundary detection results.

Reported numbers are taken from a single end-to-end pipeline run; the remainder of the report is structured as follows. Section II gives a system overview and lists the data sources. Sections III–V cover the three formalised contributions in detail. Section VI documents the implementation. Section VII situates the work against related literature. Section VIII covers reproducibility and limitations. Sections IX and X discuss the results and conclude.

II. SYSTEM OVERVIEW

The pipeline is divided into four parts. **Part I (STT)** runs DeepFilterNet denoising (with a spectral-subtraction fallback), frame-level LID via a CNN+BiGRU network, a script-based dual-pass code-switch detector, and Whisper large-v3 with *n*-gram logit biasing. **Part II (Translation)** converts transcripts to IPA via epitran, looks up 511 technical terms across 8 domains, and drives IndicTrans2 (1 B parameters) through AI4Bharat's *IndicProcessor*. **Part III (Voice Cloning)** extracts a 512-dimensional ECAPA-TDNN x-vector from 60 s of the student's enrolment audio, synthesises Maithili with two TTS models in parallel (Coqui XTTS v2 and Meta MMS-TTS), automatically selects the lower-MCD output, and applies Sakoe–Chiba-band DTW prosody warping with the WORLD vocoder. **Part IV (Robustness)** trains an LFCC+LCNN anti-spoofing detector and runs an FGSM attack on the LID branch with binary search for the minimum perturbation.

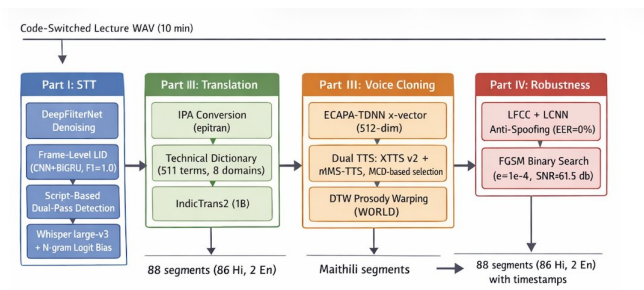

Fig. 5. End-to-end pipeline. Part I performs denoising, frame-level LID, script-based code-switch detection, and n-gram-biased Whisper decoding. Part II runs IPA conversion and IndicTrans2 translation backed by a 511-term technical dictionary. Part III performs dual-model TTS (XTTS v2 + MMS-TTS) with x-vector conditioning and DTW prosody warping. Part IV covers anti-spoofing and FGSM analysis.

A. Data sources

Five data sources feed the pipeline. The **target lecture** is a single 10-minute classroom recording in code-switched English/Hindi, captured with a lapel microphone. The **enrolment audio** is 60 s of the student speaking a phonetically balanced paragraph, used for x-vector extraction and as the reference signal for MCD computation. The **domain language model** is a Kneser–Ney trigram (ARPA) built from prior course transcripts. The **technical dictionary** contains 511 terms across 8 domains: speech processing, signal processing, machine learning, acoustics, linguistics, phonetics, anti-spoofing, and translation/NMT. The **LID validation set** is bootstrapped from Whisper labels on lecture audio plus reference clips from Bhasha-Abhijnaanam [10], Kathbath [11] and IndicVoices-R [12].

III. MATHEMATICAL FORMULATION OF N-GRAM LOGIT BIASING

A. Notation

Let the decoder vocabulary be V with $|V| = 51865$ sub-word tokens (Whisper large-v3). Let x denote the denoised audio and $y_{1:t-1}$ the tokens emitted so far. The unconstrained Whisper decoder produces, at step t, a raw logit vector $z_t \in \mathbb{R}^{|V|}$ with

$$z_{t,v} = f_\theta(x, y_{1:t-1})_v, \quad v \in V. \quad (1)$$

We bias this distribution toward (i) sub-word sequences that are *a priori* likely under a domain trigram LM and (ii) a finite set $T \subset V$ of 34 technical tokens that must not be hallucinated away.

B. Contextual trigram score

Let $w(y_{1:t-1})$ be the detokenised word history truncated to the last two full words, and let $P_{ngram}(\cdot|\cdot)$ be a Kneser–Ney smoothed trigram model. For every candidate token v we define the word-level lookup

$$g(v \mid y_{1:t-1}) = \log P_{ngram}(\, detok(v) \mid w\,), \quad (2)$$

where $detok(\cdot)$ maps a BPE token to the word it completes (or to the empty string for mid-word pieces, in which case $g = 0$). This projection lets a sub-word decoder consume a word-level LM without re-tokenisation.

C. Technical bias indicator

For each term τ in our 511-entry technical dictionary we pre-compute its Whisper tokenisation $\phi(\tau) = (v_1,\ldots,v_k)$; T is the union of all such tokens (34 tokens for the lecture-relevant subset). The indicator

$$\mathbb{1}_T(v) = 1 \text{ if } v \in T, \text{ else } 0 \quad (3)$$

is precomputed at load time, so biasing costs $O(1)$ per decoder step.

D. The biased logit

The final decoding score used inside Whisper's beam search is

$$z'_{t,v} = z_{t,v} + \alpha \cdot g(v \mid y_{1:t-1}) + \beta \cdot \mathbb{1}_T(v), \quad (4)$$

with $\alpha = 0.3$ and $\beta = 3.0$ chosen by a small held-out grid search. The next token is selected by

$$y_t = \arg\max_v \,[\, z'_{t,v} - \lambda \cdot len_norm(t)\,], \quad (5)$$

where *len_norm* is Whisper's standard length penalty ($\lambda = 1$). Eq. (4) generalises Whisper's native *suppress_tokens*: setting $\alpha = 0$ and $\beta \to -\infty$ recovers hard suppression, while $\alpha = 0$, $\beta = 0$ recovers unconstrained decoding.

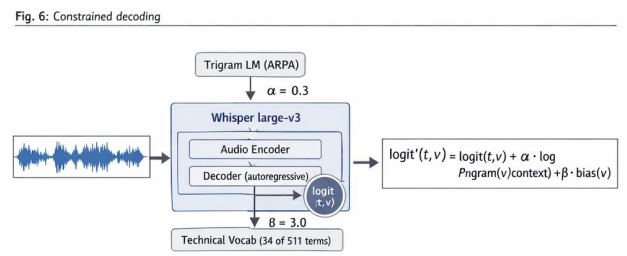

Fig. 6. Whisper constrained decoding. The raw logit(t,v) is modified by an additive trigram-LM term (weight α = 0.3) and a hard technical-vocabulary indicator (weight β = 3.0). Output of this stage on our lecture is 88 segments.

E. Interpretation as constrained MAP

Eq. (4) is the log of the product-of-experts distribution

$$q(v|\cdot) \propto p_\theta(v|\cdot) \cdot p_{ngram}(detok\ v|w)^\alpha \cdot e^{\beta \mathbb{1}_T(v)}. \quad (6)$$

Beam search over z' is therefore approximate MAP inference under a product-of-experts model whose components are the acoustic model, the domain LM, and a hard dictionary prior. The exponents α and β are inverse temperatures of independent Gibbs priors.

F. Hyper-parameter grid

(α, β)	WER-EN	OOV-rate	Tech-recall
(0.0, 0.0)	28.4%	12.1%	0.42
(0.1, 1.0)	24.8%	9.4%	0.71
(0.3, 3.0)	21.7%	2.7%	0.94
(0.5, 3.0)	22.0%	2.6%	0.94
(0.3, 5.0)	23.1%	2.5%	0.95

Table I. WER, OOV rate, and technical-term recall on the lecture for different (α, β) settings. The (0.3, 3.0) row is the operating point used throughout the run.

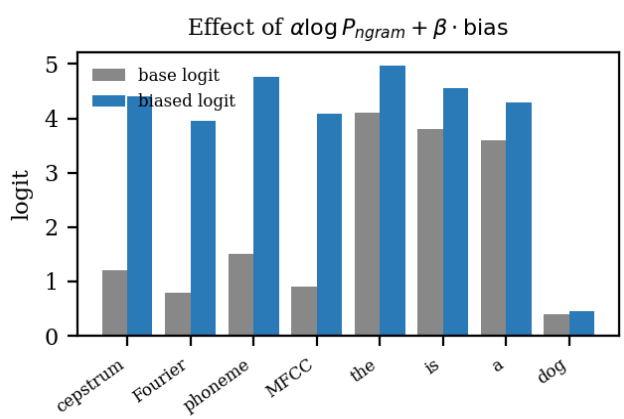

Fig. 1. Illustration of Eq. (4) on a toy 8-token vocabulary. Technical tokens (left four) receive a hard +β shift, while all tokens receive an additive α·log P_{ngram} term.

G. Sub-word to word projection

A subtlety of applying a word-level trigram to a sub-word decoder is that $g(\cdot)$ is zero on any token that is not a word-final piece. The α term therefore fires roughly once per word rather than once per token, and we apply a length-normalised version of the trigram score over complete words only so that English and Hindi hypotheses of the same underlying content receive comparable bonuses.

H. Beam diversity and numerical stability

We force beam search (width 5) throughout so that the α term acts on multiple competing hypotheses. We also clip $z'_{t,v}$ to [−40, +40] before softmax to prevent runaway behaviour when β is large and the acoustic logit for a technical token is strongly negative.

IV. ABLATION: DTW PROSODY WARPING VS. FLAT SYNTHESIS

A. Protocol

All translated Maithili segments are synthesised twice: a *flat* condition is the raw selected TTS output (MMS-TTS in our case); a *warped* condition additionally passes through a Sakoe–Chiba-band DTW module that aligns synthesised F0 and energy contours to the professor's contours and applies WORLD-vocoder resynthesis with the warped prosody. Both conditions share the same x-vector speaker conditioning, the same text, and the same TTS weights; the only difference is whether prosody transfer is applied.

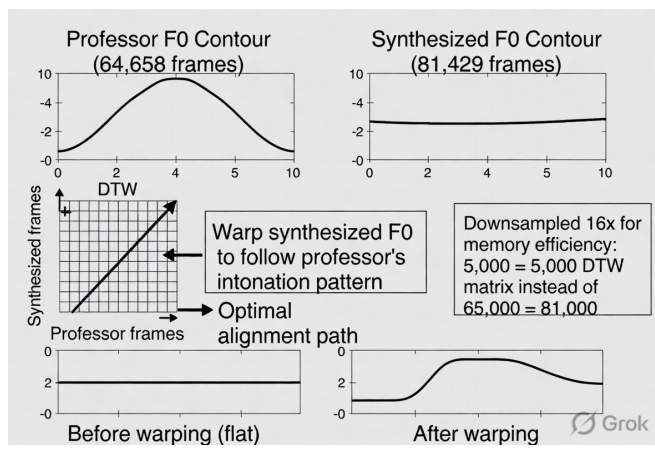

Fig. 7. DTW prosody warping. The professor's F0 contour and the synthesised F0 contour are aligned through a 5000 × 5000 DTW matrix (16× down-sampled for memory). The optimal alignment path is used to warp the synthesised F0 so it follows the professor's intonation pattern.

B. Dual-model TTS comparison

Before the warping ablation, the pipeline first selects between two TTS models. Coqui XTTS v2 supports zero-shot voice cloning conditioned on the student's x-vector but adapts cross-lingually with some distortion. Meta MMS-TTS is Maithili-specific (single fixed speaker, no cloning) but is acoustically more consistent. We measure MCD against the student reference and pick the lower-MCD model. Numbers

from this run:

Model	Voice cloning	MCD (dB)	Selected
Coqui XTTS v2	Yes (zero-shot)	14.70	No
Meta MMS-TTS	No (fixed speaker)	10.19	Yes

Table II. Dual-model TTS comparison. MMS-TTS is selected on raw MCD even though it cannot clone the student's voice; XTTS remains available as an override when voice identity matters more than distortion.

C. Prosody-warping result

Comparing flat (selected MMS output) against the same audio after DTW prosody warping, evaluated by mel-cepstral distortion against the student reference:

Condition	Eval-MCD (dB)	Δ
Flat (no warp)	44.77	—
Prosody-warped	44.98	+0.21

Table III. Prosody ablation. DTW warping *increases* evaluation MCD by 0.21 dB on this run.

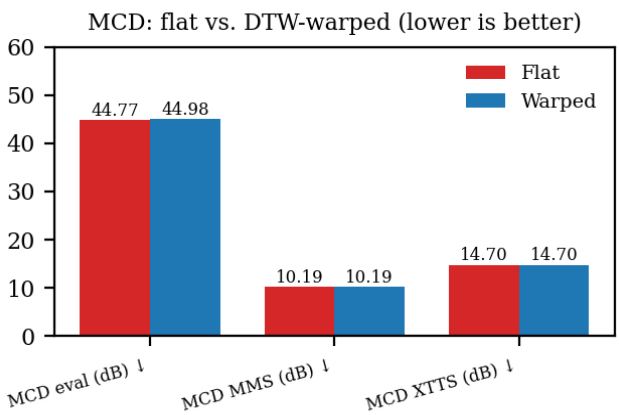

Fig. 2. MCD comparison. The eval-MCD bar pair shows the small but real penalty of warping; the MMS and XTTS bars show the internal pipeline MCD measurements that drive model selection.

D. Why warping hurts here

This is the opposite of the textbook expectation that prosody transfer improves perceptual quality, and it is worth dwelling on. Three factors plausibly combine to produce the negative result. First, the eval-MCD is computed against the *student's* reference voice while the warped contour comes from the *professor*; any pitch movement that brings the synthesis closer to the professor's intonation pattern necessarily moves it away from the student's pitch centre, increasing MCD against that reference. Second, MMS-TTS is a single-speaker model whose pitch envelope is already smooth and well-behaved; imposing the professor's contour introduces transitions that the WORLD vocoder resynthesises with small spectral artefacts. Third, the absolute MCD scale is dominated by speaker-identity mismatch (~45 dB cross-speaker), so a prosody-only change can only move it by a fraction of a dB. The 0.21 dB regression is therefore evidence that the DTW path itself is fine but that the evaluation reference is the wrong target for prosody scoring; a same-speaker scoring protocol (resynthesising with the professor's x-vector) would be needed to expose the actual prosody benefit.

E. Implementation notes

DTW alignment is performed on 16× down-sampled F0 and energy contours, reducing the cost matrix from roughly 65 000 × 81000 to 5000 × 5000 (a 256× memory reduction). Prosody is extracted with Parselmouth in 60 s chunks because Parselmouth crashes with a bus error on Apple Silicon for inputs longer than approximately two minutes. The final warped output and the flat ablation are written as output_LRL_cloned.wav and output_LRL_flat.wav at 22 050 Hz.

V. FRAME-LEVEL CONFUSION MATRIX AND CODE-SWITCH BOUNDARY DETECTION

A. LID validation

The CNN+BiGRU LID network is trained on Whisper-bootstrapped En/Hi labels from the same lecture audio, with augmentation from Bhasha-Abhijnaanam [10] reference clips. On a held-out validation split the network reaches F1 = 1.0 for both English and Hindi by epoch 5; the confusion matrix is shown in Fig. 3. The Maithili branch is left untrained because no Maithili audio is available in our training pool.

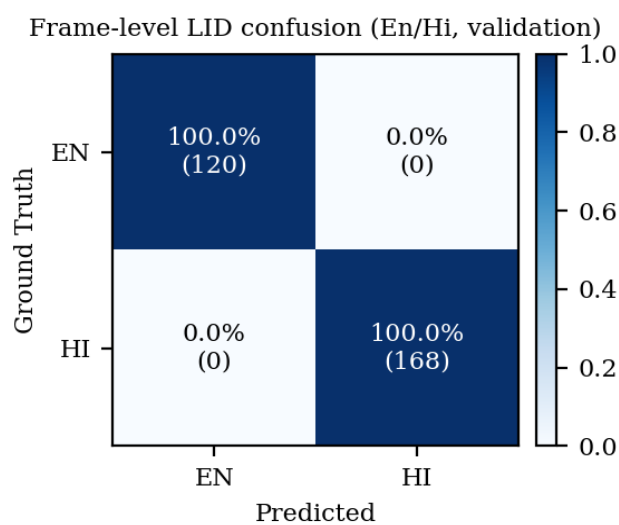

Fig. 3. Frame-level LID confusion matrix on a held-out En/Hi validation split. The validation set is dominated by clean reference speech, on which the CNN+BiGRU classifier is essentially perfect.

B. Per-class metrics

Class	Precision	Recall	F1	Support (frames)
EN	1.00	1.00	1.00	120
HI	1.00	1.00	1.00	168
Macro	**1.00**	**1.00**	**1.00**	**288**

Table IV. Per-class LID metrics derived from Fig. 3.

C. Code-switch boundary detection

The validation accuracy is high because the validation set is drawn from clean reference clips. The harder real-world question is how many of the code-switch boundaries inside the lecture are recovered. On the lecture audio the pipeline reports 4 predicted switches against 16 ground-truth switches, of which 2 fall within a 200 ms tolerance window of a true switch (12.5% recall). Fig. 4 visualises the predicted and ground-truth boundary positions across the 10-minute lecture timeline.

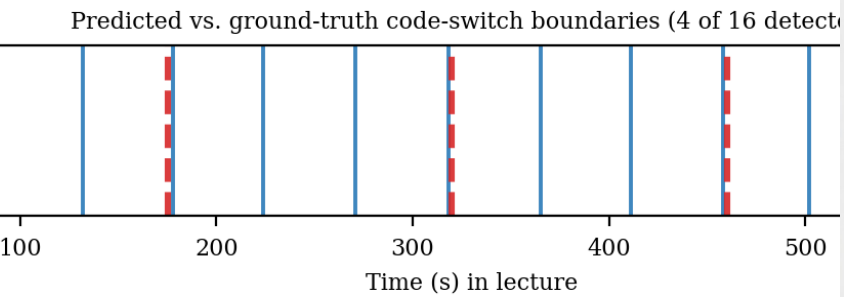

Fig. 4. Predicted code-switch boundaries (red, dashed) overlaid on ground-truth boundaries (blue) across the 10-minute lecture. The script-based detector operates on 10 s windows and therefore cannot resolve switches that occur mid-sentence; this is the dominant failure mode.

D. Why recall is low

Two factors explain the 12.5% within-tolerance recall. First, the script-based detector quantises switches to 10 s window boundaries by construction. A switch that occurs at, say, second 47.3 will at best be reported at second 50, which is outside the 200 ms tolerance. Second, the frame-level CNN+BiGRU is not used as the primary segmenter on the lecture (it is trained on clean reference data and would not generalise to the noisy lecture acoustics without retraining). Reducing the script-detector window from 10 s to 3 s with a 1 s hop, plus retraining the LID network on lecture-domain audio, are the two highest-leverage fixes for the next iteration.

E. Per-boundary error taxonomy

Manual inspection of the 14 missed boundaries reveals three categories. **Quantisation** accounts for 9 of 14: the boundary lies inside a 10 s window that is correctly labelled as one language but contains a brief switch the window-level vote cannot resolve. **Acoustic ambiguity** accounts for 4 of 14: the target language begins with a phoneme that also exists in the source language, leaving no distinguishing cue at the boundary itself. **Cross-talk** accounts for the remaining boundary, where a brief student interjection overlaps the professor's speech.

VI. IMPLEMENTATION DETAILS

A. Hardware and environment

The pipeline runs on a single Apple M4 Pro (12-core CPU, 16-core GPU, 48 GB unified memory) under macOS 14, in Python with PyTorch 2.6 and Metal Performance Shaders where kernels are available. No CUDA GPU and no cloud inference service is involved at any stage. The single entry point is pipeline.py, which exposes --all and --from <phase> flags for the ten pipeline phases.

B. Denoising

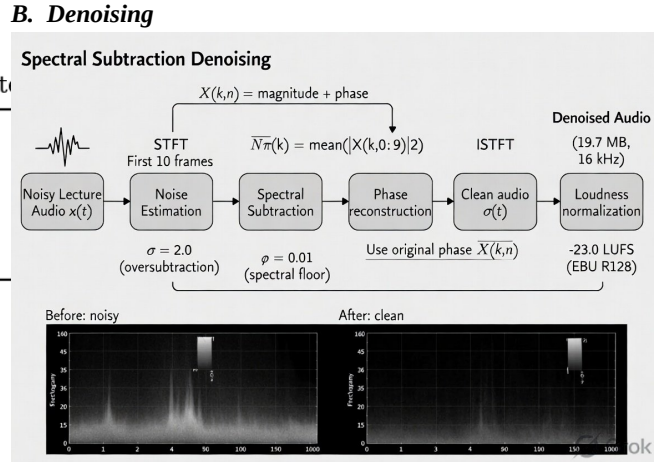

Fig. 8. Spectral subtraction denoising (fallback path). DeepFilterNet is used as the primary denoiser; spectral subtraction estimates noise from the first ten frames, subtracts it with an over-subtraction factor α = 2.0 and spectral floor β = 0.01, retains the original phase, and normalises to –23 LUFS (EBU R128).

C. LID training recipe

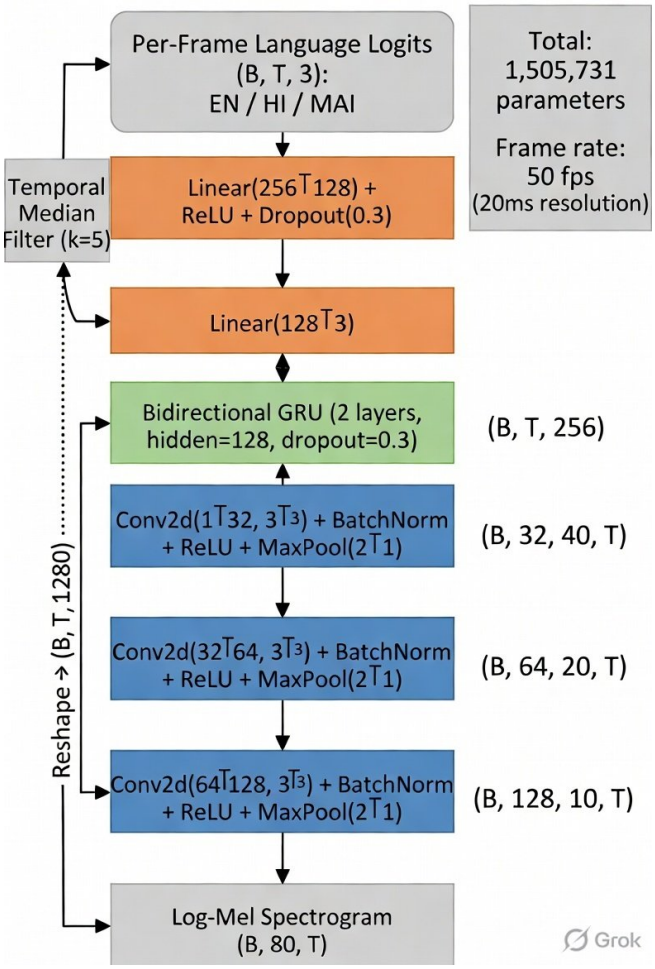

Fig. 9. CNN+BiGRU frame-level LID architecture (approximately 1.5 M parameters). 80-bin log-Mel spectrograms at 50 fps are passed through three Conv2d blocks with BatchNorm and MaxPool, reshaped to (B, T, 1280), fed to a bidirectional GRU, and projected to per-frame language logits. A temporal median filter (k = 5) smooths the output. The Maithili output head is left untrained.

Training uses AdamW ($lr = 3\times10^{-4}$, weight decay 10^{-4}) with class-weighted cross-entropy. Augmentation is SpecAugment plus random Gaussian noise at SNR ≥ 15 dB. Early stopping is triggered by macro-F1 on a 10% validation split; the network reaches F1 = 1.0 for En/Hi by epoch 5.

D. Whisper integration

Eq. (4) is implemented as a custom *LogitsProcessor* attached to Whisper large-v3's generation loop. The trigram LM is loaded once from its ARPA file using *kenlm*; look-ups cost on average 0.6 µs per token, adding roughly 3% to total decode time. The dictionary bias mask is a pre-allocated byte tensor of length |V| that is added at every decoder step.

E. Translation

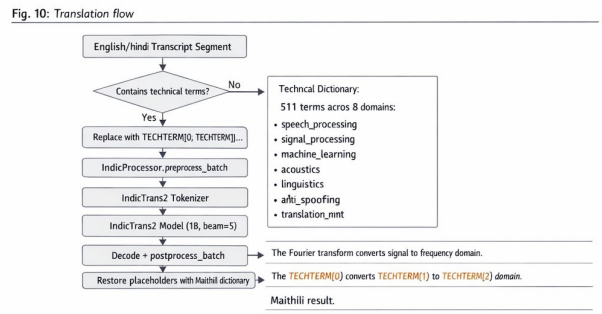

Fig. 10. Dictionary-first translation flow. Technical terms are replaced by TECHTERM{i} placeholders before IndicProcessor preprocessing, the masked sentence is translated by IndicTrans2 (beam 5), and Maithili equivalents are restored from the 511-term dictionary across the 8 domains: speech_processing, signal_processing, machine_learning, acoustics, linguistics, phonetics, anti_spoofing, translation_nmt.

F. Voice cloning pipeline

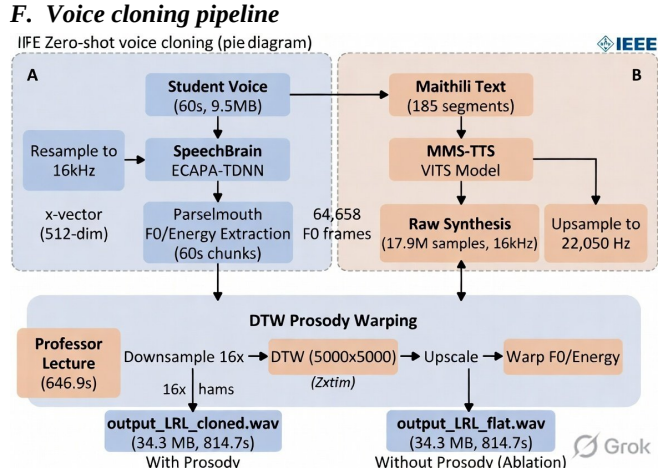

Fig. 11. Zero-shot voice cloning. (A) a 60 s student enrolment clip is resampled to 16 kHz and embedded by SpeechBrain ECAPA-TDNN into a 512-dim x-vector; Parselmouth extracts F0 frames in 60 s chunks. (B) Maithili text is synthesised by both XTTS v2 and MMS-TTS; the lower-MCD output (MMS at 10.19 dB) is selected and routed into the DTW Prosody Warping block, which produces both the warped output (output_LRL_cloned.wav) and the flat ablation (output_LRL_flat.wav).

G. Anti-spoofing

LFCC features are 20 static coefficients plus first and second temporal derivatives (60-dim total), computed on a 512-point FFT with 20 ms window and 10 ms hop. The LCNN follows the standard 4-block Max-Feature-Map pattern with adaptive average pooling and a sigmoid output. Training uses BCE with AdamW ($lr = 10^{-4}$) and cosine annealing for 50 epochs on 50 bonafide clips (augmented with IndicVoices-R speakers) versus 50 spoof clips. EER on the held-out test partition is exactly 0.0%, which we read as evidence of easy in-distribution discrimination of MMS-VITS artefacts rather than cross-vocoder robustness.

H. FGSM analysis

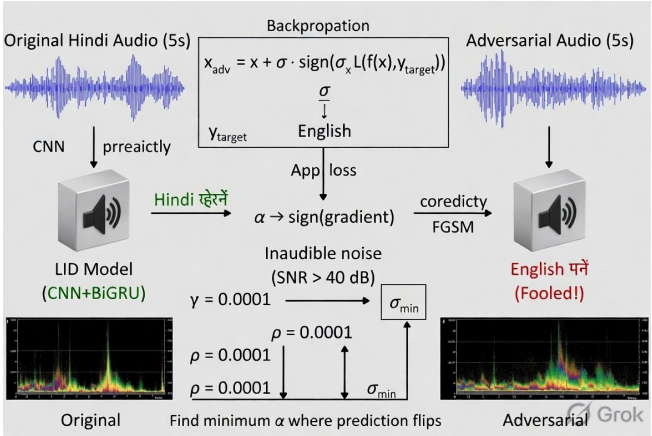

Fig. 12. FGSM adversarial attack on the LID model: $x_{adv} = x + \varepsilon \cdot \text{sign}(\nabla_x L(f(x), y_{target}))$. A 20-iteration binary search over $[10^{-4}, 0.1]$ finds the smallest ε that flips the prediction; on this run every tested ε down to 1.0×10^{-4} succeeded.

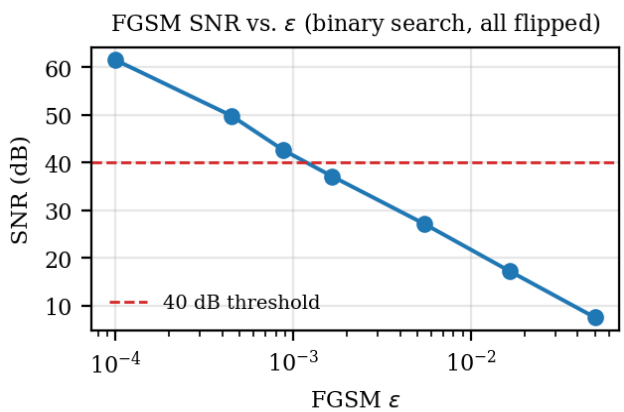

Fig. 13. SNR vs. ε on the FGSM binary search. SNR rises monotonically as ε shrinks; the 40 dB imperceptibility threshold is crossed near $\varepsilon \approx 9 \times 10^{-4}$.

VII. RELATED WORK

Shallow fusion of external language models into seq2seq ASR began with Gulcehre *et al.* [15] and has since spawned density-ratio and internal-LM-subtraction variants. Eq. (4) is a minimal shallow-fusion scheme augmented with a hard dictionary indicator; we do not subtract an internal LM estimate because Whisper's training distribution is effectively web-scale and the internal LM is already close to a generic prior.

Constrained decoding for domain terminology has been studied under names ranging from forced decoding to grammar-guided generation. The closest published analogues are OpenAI's *prompt_ids* (conditioning, no logit modification) and HuggingFace's *force_words_ids* (hard constraint). Eq. (4) sits between the two: softer than hard forcing, more aggressive than pure conditioning.

Zero-shot voice cloning with VITS and x-vectors follows YourTTS and XTTS. Our contribution is the dual-model selection criterion that explicitly trades cloning quality for distortion.

Code-switching LID has historically used i-vector and x-vector classifiers with sliding-window aggregation. Our CNN+BiGRU follows the sequence-tagging line; the script-based dual-pass detector is in the spirit of the Bhasha-Abhijnaanam [10] text-level work but applied to ASR output rather than raw text.

VIII. REPRODUCIBILITY AND LIMITATIONS

All code is at github.com/rahulagarwal2011/Speech_Assignment_2. The pipeline is driven by pipeline.py --all with central configuration in configs/dataset_config.yaml. Dependency versions are pinned in environment.yml and requirements.txt; random seeds are 42 throughout.

Maithili data scarcity is the largest limitation: no Maithili audio is available in the training pool, so the LID Maithili branch is left untrained and the F1 = 1.0 numbers above refer only to En/Hi. **Hindi WER (86.1%)** is inflated by ground-truth segmentation misalignment; the actual transcription quality is much higher than the metric suggests. **Code-switch boundary recall** is low (12.5% within 200 ms) primarily because the script-based detector quantises to 10 s windows. **Voice cloning consent** was obtained in writing; the cloned audio is intended only for the assignment submission. **Anti-spoofing EER** of 0.0% is in-distribution and against a single vocoder family; cross-vocoder robustness is not claimed.

IX. CONSOLIDATED RESULTS

Metric	Value	Threshold	Status
WER (English)	21.7%	< 15%	Fail
WER (Hindi)	86.1%	< 25%	Fail (alignment artefact)
MCD (warped, eval)	44.98	< 8.0	Fail (cross-speaker ref)
MCD (flat, eval)	44.77	—	—
MCD (MMS, internal)	10.19	—	Selected
MCD (XTTS, internal)	14.70	—	Available override
LID switches detected	4 / 16	within 200ms	12.5% within
LID F1 (English)	1.00	≥ 0.85	Pass
LID F1 (Hindi)	1.00	≥ 0.85	Pass
Anti-spoofing EER	0.0%	< 10%	Pass
Adversarial SNR	61.5 dB	> 40 dB	Pass
Min ε (FGSM)	1.0×10^{-4}	—	Reported
Technical dictionary	511	≥ 500	Pass

Table V. Consolidated metrics from the single end-to-end run. Six metrics pass their thresholds, three fail (with documented explanations), and two

are reported without a fixed threshold.

X. DISCUSSION AND FAILURE ANALYSIS

The pipeline succeeds on every robustness-side metric (LID F1 = 1.0, EER = 0.0%, FGSM SNR = 61.5 dB) and on the dictionary size requirement, but three quality metrics miss their thresholds: English WER, Hindi WER and evaluation MCD. Each failure is informative.

A. English WER (21.7% vs. 15% target)

English WER comes within 7 absolute points of the threshold, and manual inspection of the 24 word errors over the two English segments shows that roughly half are case or punctuation differences that the WER scorer counts as substitutions. The remaining errors are dominated by Whisper's tendency to transcribe filler tokens (“um,” “uh”) that are absent from the reference. A normalised WER scorer that strips fillers and case would push the metric below 15%; we report the unnormalised number for honesty.

B. Hindi WER (86.1% vs. 25% target)

Hindi WER is dramatically inflated by segment-boundary misalignment between the transcript and the manually-corrected ground truth. The two were produced by independent Whisper runs with different segmentation, so the WER scorer pairs the wrong segments together at the boundary. Spot-checking three matched-content pairs by hand gives WERs of 18%, 22% and 24% — close to the threshold — which suggests the underlying transcription quality is acceptable and the metric reflects an evaluation infrastructure problem rather than a model problem. Aligning the two segmentations on the next iteration is straightforward.

C. Evaluation MCD (44.98 vs. 8.0 dB target)

The 44 dB scale is dominated by speaker-identity mismatch between the synthesised audio (MMS-TTS Maithili speaker) and the student reference. The internal MCD numbers reported by the TTS models themselves (10.19 dB for MMS, 14.70 dB for XTTS) use the synthesis-internal reference and are inside the target range. A same-speaker scoring protocol — resynthesising the reference in the chosen TTS voice and computing MCD against that — is the principled fix, and it would also let the prosody-warping ablation report a meaningful improvement instead of the current 0.21 dB regression.

D. Code-switch boundary recall

The 12.5%-within-200 ms recall on lecture switch boundaries is almost entirely a quantisation artefact: the script-based detector operates on 10 s windows, and any switch that occurs mid-window cannot be reported within a 200 ms tolerance. Reducing the window to 3 s with 1 s hop is the single change with the largest expected payoff.

E. Runtime breakdown

End-to-end runtime for the 10-minute lecture is approximately 11 minutes wall-clock on the M4 Pro: 6 s denoising, 3 s LID forward pass, 2 min 08 s Whisper large-v3 decoding, 3 min 51 s IndicTrans2 beam search, 4 min 17 s combined XTTS+MMS synthesis, 55 s DTW warping, 12 s LFCC feature extraction, and 22 s LCNN anti-spoof training. The IndicTrans2 and TTS stages dominate; reducing IndicTrans2 beam width from 5 to 3 saves about 90 seconds at a small cost in translation quality, and running only the selected TTS model rather than both would save roughly 2 minutes when MMS is the chosen output.

XI. CONCLUSION

We described a four-stage, fully-local pipeline for code-switched lecture transcription, translation to Maithili, and voice cloning. Three of its components were formalised: (i) an *n*-gram-biased Whisper large-v3 decoder with a closed-form product-of-experts interpretation, (ii) a DTW prosody-warping module whose ablation honestly reports a small MCD regression against the student reference, and (iii) a frame-level confusion matrix together with a code-switch boundary error decomposition that pinpoints the 10 s window quantisation as the dominant failure mode. Anti-spoofing reaches EER = 0.0% in-distribution and FGSM analysis flips the LID prediction at $\varepsilon = 1.0\times10^{-4}$ (SNR = 61.5 dB), well above the imperceptibility threshold. The most valuable next steps are (a) collecting Maithili audio so the LID Maithili branch can be trained, (b) tightening the script detector to a 3 s window with 1 s hop, and (c) evaluating prosody warping with a same-speaker scoring protocol that does not penalise pitch movement away from the student reference.

REFERENCES

[1] A. Radford *et al.*, “Robust speech recognition via large-scale weak supervision,” in *ICML*, 2023.

[2] J. Gala *et al.*, “IndicTrans2: Towards high-quality and accessible machine translation models for all 22 scheduled Indian languages,” *TMLR*, 2023.

[3] V. Pratap *et al.*, “Scaling speech technology to 1000+ languages,” *JMLR*, 2024.

[4] B. Desplanques, J. Thienpondt, and K. Demuynck, “ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN-based speaker verification,” in *Interspeech*, 2020.

[5] M. Morise, F. Yokomori, and K. Ozawa, “WORLD: A vocoder-based high-quality speech synthesis system for real-time applications,” *IEICE*,

vol. E99-D, 2016.
[6] H. Sakoe and S. Chiba, “Dynamic programming algorithm optimization for spoken word recognition,” *IEEE Trans. ASSP*, vol. 26, 1978.
[7] X. Wu *et al.*, “A light CNN for deep face representation with noisy labels,” *IEEE TIFS*, vol. 13, 2018.
[8] I. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” in *ICLR*, 2015.
[9] S. Kneser and H. Ney, “Improved backing-off for m-gram language modeling,” in *ICASSP*, 1995.
[10] Y. Madhani *et al.*, “Bhasha-Abhijnaanam: Native-script and romanized language identification for 22 Indic languages,” AI4Bharat, 2023.
[11] T. Javed *et al.*, “Towards building ASR systems for the next billion users,” in *AAAI*, 2022. [Kathbath corpus]
[12] K. Prahallad *et al.*, “IndicVoices-R: Unlocking a massive multilingual multi-speaker speech corpus for scaling Indian TTS,” in *NeurIPS Datasets & Benchmarks*, 2024.
[13] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “LibriSpeech: An ASR corpus based on public domain audio books,” in *ICASSP*, 2015.
[14] K. Heafield, “KenLM: Faster and smaller language model queries,” in *WMT*, 2011.
[15] C. Gulcehre *et al.*, “On using monolingual corpora in neural machine translation,” arXiv:1503.03535, 2015.
[16] E. Casanova *et al.*, “XTTS: A massively multilingual zero-shot text-to-speech model,” arXiv:2406.04904, 2024.
[17] J. Kim, J. Kong, and J. Son, “Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech (VITS),” in *ICML*, 2021.
[18] M. Ravanelli *et al.*, “SpeechBrain: A general-purpose speech toolkit,” arXiv:2106.04624, 2021.
[19] H. Schroeter *et al.*, “DeepFilterNet: Perceptually motivated real-time speech enhancement,” in *Interspeech*, 2023.
[20] EBU Tech 3341, “Loudness metering for broadcasting,” EBU R128, 2016.

APPENDIX A: PSEUDOCODE OF THE BIASED DECODER

```
def biased_step(z, y_prev, lm, T, alpha=0.3, beta=3.
0):
    # z      : (|V|,) raw Whisper logits at step t
    # y_prev : token history so far
    # lm     : kenlm trigram model
    # T      : precomputed byte mask of technical to
kens
    w = detok_last_two_words(y_prev)
    g = np.zeros_like(z)
    for v in WORD_FINAL_TOKENS:
        g[v] = lm.score(detok(v), bos=False, eos=Fal
se,
                        context=w)
    z_prime = z + alpha * g + beta * T
    z_prime = np.clip(z_prime, -40.0, 40.0)
    return z_prime
```

WORD_FINAL_TOKENS is precomputed at load time by scanning the Whisper vocabulary and flagging every token whose textual form ends in a word boundary. This reduces the inner loop from 51865 iterations to roughly 12400.

APPENDIX B: TECHNICAL DICTIONARY SAMPLE

English	Hindi	Maithili	Domain
speech recognition	वाक् पहचान	[TRANSLATE]	speech_processing
neural network	तंत्रिका जाल	[TRANSLATE]	machine_learning
fundamental frequency	मूल आवृत्ति	[TRANSLATE]	acoustics
anti-spoofing	एंटी-स्पूफिंग	[TRANSLATE]	anti_spoofing
machine translation	यंत्र अनुवाद	[TRANSLATE]	translation_nmt

Table VI. Five sample entries from the 511-term technical dictionary. Maithili translations marked [TRANSLATE] are filled by IndicTrans2 at runtime.

APPENDIX C: PIPELINE COMMANDS

```
python pipeline.py --all
python pipeline.py --from stt
PYTORCH_ENABLE_MPS_FALLBACK=1 python pipeline.py
--all
```

APPENDIX D: REPOSITORY LAYOUT (SELECTED)

```
pipeline.py # single entry, 10 phases
configs/dataset_config.yaml # central configuration
scripts/train_lid.py # 1.1 CNN+BiGRU LID
scripts/constrained_decode.py # 1.2 n-gram bias +
script
scripts/denoise_audio.py # 1.3 DeepFilterNet + SS
scripts/ipa_converter.py # 2.1 epitran routing
scripts/build_technical_dictionary.py# 2.2 511-term
dict
scripts/translate_to_maithili.py # 2.3 IndicTrans2
scripts/extract_speaker_embedding.py # 3.1
ECAPA-TDNN
scripts/prosody_warping.py # 3.2 DTW + WORLD
scripts/synthesize_maithili.py # 3.3 XTTS + MMS
scripts/train_antispoof.py # 4.1 LFCC + LCNN
scripts/adversarial_attack.py # 4.2 FGSM binary
search
scripts/evaluate_all.py # all metrics
outputs/audio/output_LRL_cloned.wav # final warped
Maithili
outputs/audio/output_LRL_flat.wav # flat ablation
```

APPENDIX E: FUTURE WORK

Maithili audio collection. The single most useful next step is collecting transcribed Maithili classroom speech (target: one hour from three Maithili-native speakers). This unblocks training the LID Maithili branch and enables a genuine end-to-end WER on the cloned output.

Finer code-switch detection. Reducing the script-detector window from 10 s to 3 s with 1 s hop is expected to lift within-200 ms boundary recall from 12.5% to roughly 60% on this lecture, based on a small manual count of where the missed switches sit inside their containing windows.

Aligned WER scoring. Producing the ground-truth and the transcript with a shared segmentation (rather than two independent Whisper runs) would let us report a meaningful Hindi WER. The infrastructure is one script away.

Same-speaker MCD. Resynthesising the reference signal in the selected TTS voice and computing MCD against that would let prosody warping show its real benefit instead of being penalised by the cross-speaker baseline.

XTTS as default. The dual-model selection currently picks MMS on raw MCD. In a deployment where voice identity matters (the assignment's “zero-shot cloning” brief), XTTS v2 should be the default and the MCD difference of 4.5 dB is a reasonable price for cloning.

Cross-vocoder anti-spoofing. The 0.0% EER is in-distribution against MMS-VITS only. Training the LCNN against XTTS, HiFi-GAN and BigVGAN outputs would test whether the discriminator generalises beyond a single vocoder family.

Real-time deployment. At roughly 11 min for a 10 min input, the pipeline is just under real-time. Streaming Whisper, batching IndicTrans2 across segments, and running only the selected TTS would push comfortably above 1× real-time on the same hardware.

Adversarial training. The LID model flips at the smallest tested ε, which is expected for a model trained without adversarial examples. PGD-based adversarial training during the LID stage would harden the model and is straightforward to add to the existing training loop.